

FusionOS Nexus Installation Guide

A Minimal ARM64 Kernel for Mobile Devices

Version: 0.1.0

Last Updated: January 2026

Repository: <https://github.com/keniogkennigen-png/FusionOS-Nexus>

Table of Contents

1. [Introduction](#)
 2. [System Requirements](#)
 3. [Toolchain Installation](#)
 4. [Repository Setup](#)
 5. [Building the Kernel](#)
 6. [Running in QEMU](#)
 7. [Debugging with GDB](#)
 8. [Troubleshooting](#)
 9. [Next Steps](#)
-

Introduction

This guide provides step-by-step instructions for setting up FusionOS Nexus on your development system. FusionOS Nexus is a bare-metal operating system kernel written in C and ARM64 assembly, designed for educational purposes and demonstrating real operating system concepts for mobile devices.

The kernel targets ARM Cortex-A72 and Cortex-A53 processors commonly found in mobile devices. While the kernel is designed for real hardware, it can be developed and tested using cross-compilation and the QEMU emulator without requiring physical hardware.

Before proceeding with the installation, ensure you have reviewed the legal disclaimer in the README.md file. This software is provided for educational purposes and should not be deployed in production environments or critical systems.

System Requirements

The development system should meet the following minimum requirements for successful kernel development and testing. These requirements ensure adequate resources for cross-compilation, emulation, and debugging operations.

Hardware Requirements

Component	Minimum	Recommended
RAM	2 GB	4 GB or more
Disk Space	500 MB	1 GB or more
CPU	Any x86-64	Multi-core processor
Architecture	x86-64	x86-64 (ARM64 development via cross-compilation)

Operating System Requirements

The kernel can be built on any Linux distribution that supports the ARM64 cross-compilation toolchain. The following distributions have been tested and are recommended:

- Ubuntu 20.04 LTS or later
- Debian 11 (Bullseye) or later
- Fedora 35 or later
- Arch Linux (rolling release)

Other Linux distributions should work but may require adjusting package manager commands.

Required Software

The following software packages are required for building and testing FusionOS Nexus:

- **GCC ARM64 Cross-Compiler:** For compiling C and assembly sources into ARM64 executables
- **GNU Binutils:** For linker, assembler, and object file utilities
- **Make:** Build automation tool
- **Git:** Version control for cloning and managing the repository
- **QEMU System ARM:** ARM processor emulation for testing without physical hardware
- **GDB Multi-Arch:** Debugger with ARM64 support

Toolchain Installation

This section provides detailed instructions for installing all required software components on various Linux distributions.

Ubuntu and Debian

Open a terminal and execute the following commands to install the required packages:

```
sudo apt-get update
sudo apt-get install -y gcc-aarch64-linux-gnu binutils-aarch64-linux-
gnu \
    make git qemu-system-arm gdb-multiarch
```

The installation process may take several minutes depending on your internet connection speed. The package manager will automatically resolve and install all dependencies.

Fedora

Execute the following commands on Fedora-based distributions:

```
sudo dnf install -y gcc-aarch64-linux-gnu binutils-aarch64-linux-gnu \
    make git qemu-system-arm arm-none-eabi-gdb
```

Arch Linux

Execute the following commands on Arch Linux:

```
sudo pacman -Syu
sudo pacman -S aarch64-linux-gnu-gcc aarch64-linux-gnu-binutils \
    make git qemu-arm gdb
```

Verifying the Installation

After installation, verify that all tools are correctly installed by checking their versions:

```
# Verify cross-compiler installation
aarch64-linux-gnu-gcc --version

# Expected output should show version information for GCC
# Example: aarch64-linux-gnu-gcc (GCC) 11.2.0

# Verify QEMU installation
qemu-system-aarch64 --version

# Verify GDB installation
aarch64-linux-gnu-gdb --version
```

If any of these commands fail, the corresponding package was not correctly installed. Review the installation steps and ensure all packages were installed without errors.

Repository Setup

Clone the FusionOS Nexus repository using Git and navigate to the project directory:

```
# Clone the repository
git clone https://github.com/keniogkennigen-png/FusionOS-Nexus.git

# Navigate to the project directory
cd FusionOS-Nexus
```

Configure Git identity if you have not already done so. This information is used for commit authorship:

```
# Set your name for commits
git config --global user.name "Your Name"

# Set your email for commits
git config --global user.email "your.email@example.com"
```

Verify the repository contents by listing the files:

```
ls -la
```

You should see the following files and directories:

- **kernel/** - Directory containing kernel source code
 - **Makefile** - Build configuration file
 - **linker.ld** - Linker script for memory layout
 - **README.md** - Project documentation
-

Building the Kernel

This section describes how to build the FusionOS Nexus kernel from source code.

Clean Build

If you have built the kernel before or suspect build artifacts may be corrupted, start with a clean build:

```
make clean
```

This command removes all compiled object files and build artifacts, ensuring a fresh build.

Compile the Kernel

Execute the make command to build the kernel:

```
make
```

The build process involves several stages:

1. **Assembly Compilation:** The boot stub (boot.S) is assembled into object code
2. **C Compilation:** The kernel source files (kernel.c) are compiled
3. **Linking:** Object files are linked together using the linker script
4. **Binary Generation:** The final kernel binary is created

Build Output

After a successful build, the following files are created:

File	Description
kernel.elf	ELF executable with debug symbols, suitable for GDB debugging
kernel.bin	Raw binary kernel image, suitable for direct hardware execution

File	Description
kernel.map	Linker map file showing memory layout and symbol addresses

Verify the build output:

```
ls -lh kernel.elf kernel.bin kernel.map
```

Typical file sizes are:

- kernel.elf: Approximately 200-300 KB (includes debug information)
- kernel.bin: Approximately 100-200 KB (raw binary)
- kernel.map: Approximately 10-20 KB (text file)

Build Options

The Makefile supports several build targets:

```
# Build kernel only
make

# Build with verbose output
make V=1

# Generate disassembly
make disasm

# View symbol table
make symbols

# Show kernel sections
make sections
```

Running in QEMU

QEMU provides ARM64 hardware emulation, allowing you to test the kernel without physical hardware. This section describes how to run FusionOS Nexus in QEMU.

Basic Execution

Launch QEMU with the ARM64 virt machine:

```
make qemu
```

This command starts QEMU with the following configuration:

- **Machine:** virt (ARM Virtual Machine)
- **CPU:** Cortex-A57
- **Memory:** 1 GB
- **Cores:** 2
- **Serial:** stdio (kernel output to terminal)
- **Graphics:** none (headless mode)

The kernel will initialize and begin executing. Expected output includes kernel initialization messages, scheduler startup, and idle task execution. The kernel runs in an infinite loop, executing the idle task when no other processes are ready.

Exiting QEMU

To exit QEMU, press `Ctrl+A`, then `X`. This key combination sends a quit signal to QEMU.

Expected Output

When running successfully, you should see output similar to:

```
FusionOS Nexus Kernel initialized
Memory manager initialized
Scheduler initialized
Idle task started
```

If the kernel appears to hang or crash, refer to the troubleshooting section.

Graphical Mode

For kernels with framebuffer support, you can run QEMU with graphical output:

```
make qemu-gui
```

Note: The current kernel implementation focuses on serial console output for debugging.

Debugging with GDB

GDB provides powerful debugging capabilities for kernel development. This section describes how to debug FusionOS Nexus using GDB.

Starting QEMU with Debug Support

Start QEMU in debug mode, which pauses at startup and waits for a GDB connection:

```
make qemu-debug
```

QEMU will display a message indicating it is waiting for a GDB connection on TCP port 1234.

Connecting GDB

In a separate terminal, start GDB and connect to QEMU:

```
aarch64-linux-gnu-gdb kernel.elf
```

Within GDB, establish the connection to QEMU:

```
(gdb) target remote localhost:1234
```

Debugging Commands

Common GDB commands for kernel debugging:

Command	Description
<code>break function_name</code>	Set a breakpoint at a function
<code>continue</code>	Resume execution
<code>step</code>	Step one instruction
<code>next</code>	Step one source line
<code>print variable</code>	Print variable value
<code>backtrace</code>	Show call stack
<code>info registers</code>	Show register values

Command	Description
<code>x/32x address</code>	Examine memory (32 words)

Example Debugging Session

```
# Set a breakpoint at kernel_main
(gdb) break kernel_main

# Continue execution
(gdb) continue

# Breakpoint hit, examine registers
(gdb) info registers

# Step through initialization
(gdb) next

# Print kernel state
(gdb) print current_task
```

Kernel Symbol Table

The `kernel.elf` file contains debug symbols that enable source-level debugging. Ensure you are debugging `kernel.elf` and not `kernel.bin`, as the binary file lacks debug information.

Troubleshooting

This section addresses common issues and their solutions.

Build Failures

Issue: Cross-compiler not found

```
make: aarch64-linux-gnu-gcc: Command not found
```

Solution: Install the ARM64 cross-compiler toolchain. See the Toolchain Installation section.

Issue: Missing header files or dependencies

```
fatal error: kernel.h: No such file or directory
```

Solution: Ensure you are in the project directory and the kernel directory contains kernel.h. Clone the repository again if files are missing.

QEMU Issues

Issue: QEMU fails to start

```
qemu-system-aarch64: Unsupported machine type
```

Solution: Use the correct machine type. The kernel requires the 'virt' machine type:

```
qemu-system-aarch64 -machine virt ...
```

Issue: Kernel hangs during boot

Solution: This may indicate a memory allocation failure or page table error. Enable debug output and check for error messages on the serial console. Ensure sufficient system resources are available.

Debugging Issues

Issue: GDB cannot connect to QEMU

```
Remote connection refused
```

Solution: Ensure QEMU is running in debug mode (`make qemu-debug`) and the firewall is not blocking port 1234.

Issue: No debug symbols in GDB

```
No symbol table is loaded
```

Solution: Load kernel.elf, which contains debug information:

```
(gdb) symbol kernel.elf
```

Performance Issues

Issue: QEMU runs slowly

Solution: QEMU emulation has inherent performance overhead. Ensure hardware virtualization (KVM) is enabled if available. On Linux, check that KVM is available:

```
ls /dev/kvm
```

Next Steps

After successfully building and running FusionOS Nexus, consider the following activities to deepen your understanding:

Explore the Code

Study the kernel source code to understand the implementation:

- **kernel/boot.S:** Review the ARM64 boot assembly code
- **kernel/kernel.c:** Examine the core kernel implementation
- **kernel/kernel.h:** Understand the data structures and interfaces

Modify and Extend

Experiment with the kernel by modifying parameters and adding features:

- Adjust scheduler time quantum in the kernel source
- Add new system calls following the existing patterns
- Implement additional device drivers

Test on Hardware

Deploy the kernel to real ARM64 hardware:

- Raspberry Pi 4 or 3 (with appropriate bootloader)
- ARM development boards with UART access
- QEMU's ARM64 virt machine (already tested)

Contribute

Contributions are welcome:

- Report bugs with detailed reproduction steps
- Submit patches for bug fixes or features
- Improve documentation and examples

Additional Resources

- **GitHub Repository:** <https://github.com/keniogkennigen-png/FusionOS-Nexus>
 - **ARM Architecture Reference Manual:** ARMv8-A architecture specifications
 - **QEMU Documentation:** <https://www.qemu.org/docs/master/>
 - **GDB Documentation:** <https://sourceware.org/gdb/documentation/>
-

Legal Notice

BEFORE USING FUSIONOS NEXUS, PLEASE READ THE LEGAL DISCLAIMER IN THE README.md FILE. THIS SOFTWARE IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND. THE DEVELOPERS AND CONTRIBUTORS ARE NOT LIABLE FOR ANY DAMAGES ARISING FROM THE USE OF THIS SOFTWARE.

FusionOS Nexus is an educational project demonstrating operating system concepts. It is not designed, tested, or certified for use in critical systems, production environments, or any application where failure could cause harm.

Document Version: 1.0

Last Updated: January 2026

FusionOS Nexus - Illuminating Operating System Development